

ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO (USP)

MARCELO DE OLIVEIRA PASSOS

**ANÁLISE DE DESEMPENHO DE MOTORES DE CONSULTA DE DADOS EM
SISTEMAS DE ARQUIVOS DISTRIBUÍDOS:**

Um comparativo de serviços *serverless* em *cloud computing*

SÃO PAULO

2025

MARCELO DE OLIVEIRA PASSOS

**ANÁLISE DE DESEMPENHO DE MOTORES DE CONSULTA DE DADOS EM
SISTEMAS DE ARQUIVOS DISTRIBUÍDOS:**
Um comparativo de serviços *serverless* em *cloud computing*

Monografia apresentada ao Programa de Educação
Continuada da Escola Politécnica da Universidade de São
Paulo, para obtenção do título de Especialista, pelo
Programa de Pós-Graduação em Engenharia de Dados e
Big Data.

Orientador: Prof. Me. Leandro Mendes Ferreira

SÃO PAULO

2025

FICHA CATALOGRÁFICA

Sl. No.	Particulars	Amount
1	Salaries	
2	Wages	
3	Gratuities	
4	Dividends	
5	Interest on loans	
6	Interest on deposits	
7	Interest on bonds	
8	Interest on stocks	
9	Interest on debentures	
10	Interest on mortgages	
11	Interest on other securities	
12	Interest on government securities	
13	Interest on foreign securities	
14	Interest on local securities	
15	Interest on other investments	
16	Interest on cash	
17	Interest on bank deposits	
18	Interest on other financial assets	
19	Interest on other financial liabilities	
20	Interest on other financial instruments	
21	Interest on other financial products	
22	Interest on other financial services	
23	Interest on other financial facilities	
24	Interest on other financial arrangements	
25	Interest on other financial transactions	
26	Interest on other financial operations	
27	Interest on other financial activities	
28	Interest on other financial processes	
29	Interest on other financial systems	
30	Interest on other financial networks	
31	Interest on other financial platforms	
32	Interest on other financial channels	
33	Interest on other financial interfaces	
34	Interest on other financial portals	
35	Interest on other financial gateways	
36	Interest on other financial hubs	
37	Interest on other financial nodes	
38	Interest on other financial links	
39	Interest on other financial connections	
40	Interest on other financial relationships	
41	Interest on other financial partnerships	
42	Interest on other financial alliances	
43	Interest on other financial collaborations	
44	Interest on other financial joint ventures	
45	Interest on other financial strategic investments	
46	Interest on other financial equity investments	
47	Interest on other financial debt investments	
48	Interest on other financial real estate investments	
49	Interest on other financial infrastructure investments	
50	Interest on other financial technology investments	
51	Interest on other financial innovation investments	
52	Interest on other financial research investments	
53	Interest on other financial development investments	
54	Interest on other financial social investments	
55	Interest on other financial environmental investments	
56	Interest on other financial governance investments	
57	Interest on other financial risk investments	
58	Interest on other financial compliance investments	
59	Interest on other financial security investments	
60	Interest on other financial privacy investments	
61	Interest on other financial data investments	
62	Interest on other financial analytics investments	
63	Interest on other financial automation investments	
64	Interest on other financial artificial intelligence investments	
65	Interest on other financial machine learning investments	
66	Interest on other financial blockchain investments	
67	Interest on other financial cryptocurrency investments	
68	Interest on other financial digital investments	
69	Interest on other financial online investments	
70	Interest on other financial mobile investments	
71	Interest on other financial cloud investments	
72	Interest on other financial SaaS investments	
73	Interest on other financial IaaS investments	
74	Interest on other financial PaaS investments	
75	Interest on other financial DevOps investments	
76	Interest on other financial Agile investments	
77	Interest on other financial Lean investments	
78	Interest on other financial Six Sigma investments	
79	Interest on other financial Kaizen investments	
80	Interest on other financial TQM investments	
81	Interest on other financial ISO investments	
82	Interest on other financial HACCP investments	
83	Interest on other financial GMP investments	
84	Interest on other financial GLP investments	
85	Interest on other financial GCP investments	
86	Interest on other financial GMP investments	
87	Interest on other financial GLP investments	
88	Interest on other financial GCP investments	
89	Interest on other financial GMP investments	
90	Interest on other financial GLP investments	
91	Interest on other financial GCP investments	
92	Interest on other financial GMP investments	
93	Interest on other financial GLP investments	
94	Interest on other financial GCP investments	
95	Interest on other financial GMP investments	
96	Interest on other financial GLP investments	
97	Interest on other financial GCP investments	
98	Interest on other financial GMP investments	
99	Interest on other financial GLP investments	
100	Interest on other financial GCP investments	

AGRADECIMENTOS

Agradeço à minha esposa Renata, pelo companheirismo, paciência e motivação constante, especialmente nos momentos mais difíceis dessa caminhada.

Registro também minha gratidão ao meu orientador, Prof. Me. Leandro Mendes Ferreira pela dedicação, disponibilidade e pelas orientações essenciais que contribuíram de forma decisiva para o desenvolvimento deste trabalho.

Agradeço à USP, ao programa PECE Poli e a todos os professores que, com dedicação e conhecimento, colaboraram para minha formação.

Por fim, agradeço ao meu grupo de trabalhos do curso, pela parceria, dedicação e comprometimento ao longo de toda a jornada acadêmica.

PERSPECTIVAS PROFISSIONAIS ALCANÇADAS COM O CURSO

A participação neste programa de pós-graduação representou um marco importante na minha trajetória acadêmica e profissional. Durante o curso, tive a oportunidade de aprofundar conhecimentos técnicos e estratégicos, além de desenvolver uma visão mais ampla e crítica sobre os desafios da área de engenharia de dados.

As habilidades adquiridas ao longo do programa já têm se refletido positivamente na prática profissional, contribuindo para a elaboração de soluções mais eficazes e para uma atuação mais qualificada no mercado de trabalho. Tenho convicção de que os aprendizados obtidos aqui continuarão sendo decisivos para meu desenvolvimento e para os próximos passos da minha carreira.

RESUMO

Nos últimos anos, o modelo *serverless* tem ganhado destaque por oferecer escalabilidade automática e simplificação no gerenciamento de infraestrutura, permitindo que aplicações e análises de dados sejam executadas sob demanda. A Amazon Web Services (AWS) disponibiliza diferentes serviços *serverless* voltados para consulta e processamento de dados armazenados no Amazon S3, como o EMR Serverless com Spark, o Redshift e o Athena. Cada um desses serviços possui características próprias em relação à arquitetura, ao modelo de execução e ao tipo de carga de trabalho para o qual é mais indicado. Este trabalho apresenta uma comparação entre esses três serviços, tendo como foco a análise de consultas realizadas diretamente sobre arquivos no S3, com o objetivo de compreender suas particularidades e potenciais aplicações em diferentes cenários de análise de dados.

Palavras-chave: Serverless, AWS, Big Data, S3, Athena, Redshift, EMR.

ABSTRACT

In recent years, the serverless model has gained prominence by providing automatic scalability and simplified infrastructure management, allowing applications and data analysis tasks to be executed on demand. *Amazon Web Services* (AWS) offers different serverless services designed for querying and processing data stored in *Amazon S3*, such as *EMR Serverless* with *Spark*, *Redshift*, and *Athena*. Each of these services has its own characteristics regarding architecture, execution model, and the type of workload for which it is best suited. This work presents a comparison among these three services, focusing on the analysis of queries performed directly on *S3* files, with the purpose of understanding their particularities and potential applications in different data analysis scenarios.

Keywords: Serverless, AWS, Big Data, S3, Athena, Redshift, EMR.

LISTA DE FIGURAS

<i>Figura 1 - Diagrama do modelo star schema benchmark (SSB)</i>	<i>25</i>
<i>Figura 2- Diagrama ETL EMR Serverless</i>	<i>33</i>
<i>Figura 3- Diagrama ETL Redshift Serverless</i>	<i>34</i>
<i>Figura 4- Diagrama ETL Athena</i>	<i>35</i>
<i>Figura 5 – Comparação dos tempos médios de leitura TSV + escrita Parquet por serviço</i>	<i>40</i>
<i>Figura 6– Comparação dos tempos médios de execução das consultas SSB por serviço</i>	<i>42</i>

LISTA DE TABELAS

<i>Tabela 1 - Tamanho de cada arquivo utilizado no experimento.</i>	<i>37</i>
<i>Tabela 2 – Tempo médio de leitura (TSV) + escrita (Parquet) por tabela (média de 5 amostras):</i>	<i>39</i>
<i>Tabela 3 – Tempo médio em segundos de execução das queries (Parquet):</i>	<i>41</i>

LISTA DE ABREVIACÕES

ANSI – *American National Standards Institute*
API – *Application Programming Interface*
AWS – *Amazon Web Services*
BI – *Business Intelligence*
CPU – *Central Processing Unit*
CSV – *Comma-Separated Values*
EMR – *Elastic MapReduce*
EMR Serverless – *Elastic MapReduce Serverless*
ETL – *Extract, Transform and Load*
ETL Serverless – *Extract, Transform and Load Serverless*
GPU – *Graphics Processing Unit*
IAM – *Identity and Access Management*
I/O – *Input/Output*
JSON – *JavaScript Object Notation*
MLlib – *Machine Learning Library*
MPP – *Massively Parallel Processing*
OLAP – *Online Analytical Processing*
OLTP – *Online Transaction Processing*
ORC – *Optimized Row Columnar*
RDD – *Resilient Distributed Dataset*
S3 – *Simple Storage Service*
SDK – *Software Development Kit*
SQL – *Structured Query Language*
SSB – *Star Schema Benchmark*
TPC-H – *Transaction Processing Performance Council – H*
TSV – *Tab-Separated Values*

SUMÁRIO

1. INTRODUÇÃO	20
1.1 Motivação	21
1.2 Objetivo	22
1.3 Justificativa	22
1.4 Metodologia	23
2. FUNDAMENTAÇÃO TEÓRICA.....	25
2.1 Modelagem Dimensional	25
2.2 Amazon Athena: Arquitetura e Aplicações Analíticas.....	26
2.3 Redshift Serverless: Data Warehousing Serverless	28
2.4 EMR Serverless com Apache Spark.....	28
2.4.1 Apache Spark.....	28
2.4.2 EMR Serverless	29
2.5 Validação Científica e Referencial Acadêmico.....	29
3. EXPERIMENTO	31
3.1 Ambiente Experimental e Ferramentas Utilizadas.....	31
3.2 Origem dos Dados	31
3.3 Interação com os Serviços Serverless	31
3.4 Resultados do Processamento	32
3.5 Arquitetura da Solução Utilizada	32
3.6 Implementação das ETLs e Consultas	35
3.7 Recursos e Bibliotecas Empregadas no Processo	36
3.8 Lógica de Execução das Consultas e Medição de Desempenho	38
3.8.1 Preparação e Execução Iterativa das Consultas	38
3.8.2 Envio das Consultas aos Serviços Serverless.....	38
3.8.3 Monitoramento da Execução	38

3.8.4 Cálculo do Tempo de Execução.....	38
4.RESULTADOS E COMENTÁRIOS.....	39
4.1 Transformação de Dados	39
4.2 Execução das Consultas no Formato Parquet.....	40
4.3 Resultados	42
5 CONCLUSÃO.....	44
5.1 Principais Constatações e Contribuições.....	44
5.2 Méritos do Estudo.....	44
5.3 Limitações e Restrições	45
5.4 Trabalhos Futuros	45
5.5 Contribuições para a Prática Profissional.....	46
6 REFERÊNCIAS BIBLIOGRÁFICAS	47
7 APÊNDICE A - CONSULTAS UTILIZADAS NO BENCHMARK.....	51
7.1 Consulta q1.1.....	51
7.2 Consulta q1.2.....	51
7.3 Consulta q1.3.....	51
7.4 Consulta q2.1.....	51
7.5 Consulta q2.2.....	52
7.6 Consulta q2.3.....	52
7.7 Consulta q3.1.....	52
7.8 Consulta q3.2.....	53
7.9 Consulta q3.3.....	53
7.10 Consulta q3.4.....	53
7.11 Consulta q4.1.....	53
7.12 Consulta q4.2.....	54
7.13 Consulta q4.3.....	54

1. INTRODUÇÃO

Nos últimos anos, o paradigma *serverless* emergiu como uma importante evolução na computação em nuvem, oferecendo benefícios como escalabilidade automática, otimização no uso de recursos computacionais para processamento de dados e redução da complexidade operacional para desenvolvedores. Ao eliminar a necessidade de gestão direta da infraestrutura, esse modelo facilita a implementação de aplicações analíticas e de processamento de dados, permitindo que as organizações se beneficiem de maior agilidade e eficiência (Wen et al., 2023).

Serverless refere-se a um modelo de execução em que o provedor gerencia toda a infraestrutura necessária para rodar funções desencadeadas por eventos, incluindo provisionamento, escalonamento e manutenção automática. Nessa arquitetura, o usuário concentra-se no desenvolvimento do código, enquanto a cobrança ocorre de acordo com o consumo real dos recursos computacionais. A separação entre computação e armazenamento possibilita que processamento de dados e cargas de trabalho de *Big Data* sejam executadas sob demanda, de modo flexível e eficiente (Kounev et al., 2023).

No contexto da Amazon Web Services (AWS), destacam-se várias soluções *serverless* voltadas para a análise e processamento de dados em larga escala, especialmente sobre dados armazenados em serviços de arquivos distribuídos, principalmente no serviço Amazon Simple Storage Service (AWS S3). Serviços como o Elastic MapReduce (EMR) Serverless com Spark, Amazon Redshift e Amazon Athena utilizam a linguagem *Structured Query Language* (SQL) como base para consultas e processamento de dados, facilitando a adoção por equipes analíticas. O Amazon Redshift é um *data warehouse* que executa consultas *SQL* com alta performance, inclusive utilizando extensões para consultar dados diretamente no Amazon S3 (Sen et al., 2024). O Amazon Athena permite consultas interativas via *SQL* padrão *American National Standards Institute* (ANSI) diretamente nos dados armazenados no S3, sem necessidade de infraestrutura gerenciada (Nakar, 2020). O EMR Serverless com Apache Spark suporta a execução de consultas estruturadas por meio do Spark SQL, viabilizando processamento distribuído e análise de grandes volumes de dados (Kyrychenko et al., 2025).

Cada serviço possui características próprias em relação à integração com o AWS S3, desempenho, custo e complexidade operacional, sendo indicado conforme o perfil do processamento requerido (George, 2024).

Este trabalho realiza uma comparação sistemática entre EMR Serverless, Redshift Serverless e Athena, com enfoque na execução de consultas diretamente sobre arquivos armazenados no S3. Busca-se compreender como as particularidades de cada solução influenciam o desempenho e a flexibilidade das plataformas em diferentes cenários de processamento de dados, contribuindo para a escolha mais adequada de tecnologias diante dos desafios do *Big Data* em ambientes *cloud computing*.

O foco principal deste trabalho foi a análise do desempenho das consultas do *Star Schema Benchmark* (SSB), pois esse aspecto é determinante para avaliar a eficiência das soluções serverless em cenários analíticos. A etapa de conversão dos arquivos para o formato Parquet também foi abordada, porém com menor ênfase, uma vez que essa transformação era necessária para viabilizar a execução das consultas no ambiente proposto. Assim, os resultados dessa fase foram apresentados apenas para contextualizar o processo completo, sem constituir o objetivo central da pesquisa. A escolha pelo formato Parquet está alinhada às boas práticas para otimização de consultas analíticas, pois trata-se de um formato colunar que reduz o volume de dados lidos e melhora significativamente a performance em operações de leitura e agregação.

Este trabalho não aborda a análise de custo devido à complexidade e variabilidade do modelo de cobrança dos serviços serverless da AWS, que depende de fatores como volume de dados, região e otimizações específicas. Além disso, a ausência de um cenário de produção real inviabilizaria estimativas precisas, tornando o foco na avaliação de desempenho mais adequado ao escopo do estudo. Outro fator determinante foi a limitação de tempo e recursos disponíveis para a execução do projeto, pois a análise de custo exigiria testes adicionais, coleta detalhada de métricas e integração com ferramentas de billing da AWS, o que não seria viável sem comprometer a profundidade da análise técnica.

1.1 Motivação

O modelo *serverless* tem mostrado avanços significativos em desempenho e usabilidade para o processamento de dados em larga escala em nuvem. Em termos de desempenho, a elasticidade automática e a capacidade de escalar sob demanda permitem respostas rápidas em cargas de trabalho variáveis, reduzindo latências de execução e melhorando o desempenho geral das operações de processamento de dados (WEN et al., 2023). Essas características são

essenciais para suportar *pipelines* de dados que demandam processamento eficiente de grandes volumes em tempo real ou quase real.

A usabilidade é outro ponto-chave do *serverless*, uma vez que a abstração da infraestrutura oferece simplicidade operacional e permite que engenheiros de dados concentrem-se exclusivamente na lógica e nos objetivos dos negócios, sem a complexidade do gerenciamento de servidores ou *clusters* (TOOSI et al., 2025). Ferramentas *serverless* costumam integrar-se facilmente com outros serviços, facilitando o desenvolvimento de pipelines mais eficientes e ágeis, o que potencializa a inovação e a produtividade.

Portanto, o foco no desempenho dinâmico aliado à alta usabilidade torna os serviços *serverless* propícios para enfrentar desafios complexos no processamento distribuído de dados, reduzindo barreiras técnicas e acelerando o tempo para tomada de decisões baseada em dados (SADAQAT et al., 2022).

A motivação para a realização deste trabalho decorre da necessidade prática e crescente no campo da engenharia de dados de se escolher a solução mais adequada entre os serviços *serverless* de processamento de dados analíticos, como os que são oferecidos pela AWS. Cada serviço – EMR *Serverless* com Apache Spark, Amazon Redshift e Amazon Athena – apresenta características específicas que impactam o desempenho, custo e complexidade, reforçando a importância de uma avaliação baseada em cenários reais.

1.2 Objetivo

O objetivo principal desse trabalho é comparar o desempenho de três serviços *serverless* da AWS, sendo eles: EMR *Serverless*, Redshift e Athena, por meio da execução de um conjunto padronizado de consultas de *benchmark*, avaliando o tempo de resposta de cada consulta em cada serviço, a fim de identificar suas características e eficiências relativas no processamento de dados. Desta forma, este trabalho visa fornecer um conjunto de evidências que auxiliem na tomada de decisão de escolha desses serviços em cada cenário.

1.3 Justificativa

A crescente adoção do modelo *serverless* para processamento de dados em nuvem tem gerado desafios e oportunidades para engenheiros de dados, especialmente no que tange à

otimização do desempenho e à eficiência da execução de cargas de trabalho em ambientes escaláveis (WEN et al., 2023). Embora o *serverless* ofereça facilidades no gerenciamento da infraestrutura, é crucial entender suas limitações e vantagens reais através de avaliações quantitativas rigorosas, como *benchmarks* padronizados (SADAQAT et al., 2022).

Neste contexto, comparar o desempenho de serviços *serverless* da AWS, como EMR Serverless, Redshift e Athena, permite identificar características essenciais para a escolha adequada da tecnologia conforme os diferentes perfis de carga de trabalho na engenharia de dados. Estudos recentes destacam variações de desempenho significativas entre essas plataformas, reforçando a necessidade de validação científica que apoie a tomada de decisão técnica (SCHMID et al., 2025).

Além disso, a rápida evolução das arquiteturas *serverless* demanda pesquisas contínuas para acompanhar as melhorias e entender o impacto das diferentes implementações na produtividade dos engenheiros de dados, garantindo *pipelines* mais eficientes e escaláveis. Assim, este trabalho contribuirá para preencher lacunas na literatura ao fornecer uma análise comparativa atualizada e embasada em métricas objetivas, auxiliando profissionais e pesquisadores a maximizar o potencial dessas tecnologias.

1.4 Metodologia

No contexto de avaliação de desempenho em sistemas computacionais, um *benchmark* é um conjunto padronizado de dados, consultas e métricas utilizado para medir, comparar e analisar o comportamento de diferentes tecnologias sob condições controladas. Essa abordagem garante reprodutibilidade e fornece critérios consistentes para comparação entre soluções (Gray, 1993).

Neste trabalho, foi adotado o *Star Schema Benchmark* (SSB), amplamente reconhecido na comunidade científica como um bom modelo para pesquisa em ambientes de *data warehouse* e *Online Analytical Processing* (OLAP). O SSB deriva do *Transaction Processing Performance Council* (TPC-H), um dos *benchmarks* mais tradicionais da área, porém adota um modelo dimensional em estrela em lugar do esquema altamente normalizado (*snowflake*) do TPC-H. Essa adaptação foi proposta para refletir de maneira mais realista cargas de trabalho típicas de inteligência de negócios e análises em *data warehouses* (O’Neil et al., 2009).

Sua simplicidade e foco em operações típicas de *data warehouse* o tornam ideal para avaliar motores de processamento analíticos em nuvem como Redshift Serverless, Athena e EMR Serverless na AWS, onde o armazenamento desacoplado e o processamento escalável são fundamentais para eficiência (Perach, Ronen & Kvatinsky, 2023).

Este trabalho utiliza o SSB para comparar desempenho, escalabilidade e eficiência destes serviços *serverless*, embasando-se em métricas consolidadas e práticas experimentais reconhecidas.

A escolha do SSB como metodologia de avaliação foi baseada em quatro critérios principais: (1) ampla adoção acadêmica e industrial como *benchmark* consolidada para data warehouses; (2) simplicidade e aderência ao modelo dimensional em estrela, amplamente usado no ambiente corporativo de inteligência de negócios; (3) derivação do *benchmark* tradicional TPC-H, porém simplificado e adaptado para esquemas estrela, refletindo melhor cargas reais de trabalho OLAP; e (4) compatibilidade com formatos de dados modernos, como Parquet, e ambientes *serverless* na nuvem, facilitando a realização de testes práticos em serviços como Amazon Redshift Serverless, Athena e EMR Serverless.

O SSB possui consultas representativas que simulam cargas de trabalho típicas no contexto analítico, facilitando comparações coerentes e confiáveis entre diferentes serviços e arquiteturas. Sua popularidade em pesquisas recentes, incluindo estudos na AWS e em sistemas distribuídos, o torna uma escolha adequada para *benchmarking* de soluções analíticas *serverless* na nuvem (O’Neil et al., 2009).

Além disso, a simplicidade do modelo estrela reduz a complexidade da modelagem, permitindo foco na avaliação de desempenho e escalabilidade dos serviços, sem perda de realismo nas simulações de cargas de trabalho típicas em *data warehouses* modernos. Para ilustrar a estrutura utilizada nos experimentos, apresenta-se a seguir o diagrama referente ao Star Schema Benchmark (SSB).

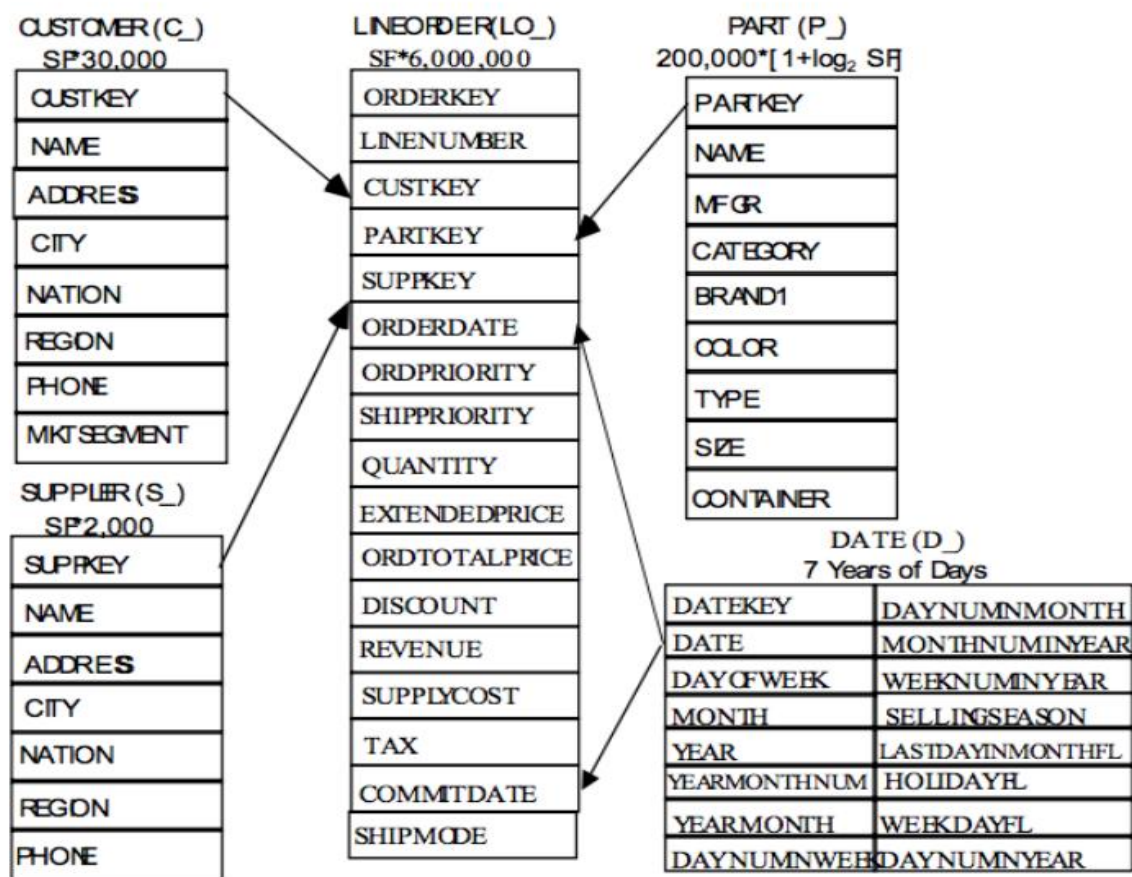


Figura 1 - Diagrama do modelo star schema benchmark (SSB)

Fonte: Sanchez, 2016

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Modelagem Dimensional

A modelagem dimensional é uma técnica de modelagem de dados amplamente utilizada para a construção de *data warehouses*, com o objetivo de facilitar a análise de dados e o suporte à decisão. Esse modelo organiza dados em estruturas que são intuitivas para os usuários de negócio, promovendo consultas rápidas e eficientes em grandes volumes de dados históricos (Parmanto, 2005). Diferentemente do modelo entidade-relacionamento tradicional utilizado em sistemas transacionais, a modelagem dimensional prioriza a desnormalização para melhorar o desempenho em consultas analíticas (Kimball; Ross, 2013).

O modelo dimensional é formado principalmente por dois tipos de tabelas: tabelas fato, que armazenam as métricas quantitativas dos processos de negócio, e tabelas dimensão, que fornecem o contexto para essas métricas, detalhando aspectos como tempo, produto, cliente ou localização (Kimball & Ross, 2013). As tabelas fato estão relacionadas às dimensões por meio de chaves estrangeiras, permitindo que as análises sejam efetuadas sob diferentes perspectivas (Hokama, 2004).

Existem dois esquemas comuns para organizar os dados no modelo dimensional: o esquema estrela (*Star Schema*) e o esquema floco de neve (*Snowflake Schema*). No esquema estrela, a tabela fato está no centro, conectada diretamente a todas as tabelas dimensão, que são desnormalizadas, facilitando consultas com poucas junções e alta performance. Já o esquema floco de neve representa uma normalização das dimensões, pois estas se conectam a subdimensões, formando uma estrutura hierárquica que reduz a redundância de dados, mas aumenta a complexidade das consultas (Schneider & Frosch-Wilke, 2011).

No *Snowflake Schema*, as dimensões principais possuem relacionamentos explícitos com suas subdimensões por meio de chaves estrangeiras, o que organiza os dados em múltiplos níveis e melhora a integridade e a qualidade dos dados (Kimball & Ross, 2013). Esta normalização traz benefícios como a economia de espaço em armazenamento e maior facilidade de manutenção dos dados, uma vez que atualizações são feitas em apenas um local. Por outro lado, tais benefícios exigem maior número de junções nas consultas, o que pode impactar negativamente o desempenho em ambientes de análises complexas (Elizabeth, 2008).

A normalização presente no esquema floco de neve auxilia a redução da redundância de dados, melhora a integridade referencial e promove um uso mais eficiente do armazenamento, principalmente em grandes volumes de dados (Schneider & Frosch-Wilke, 2011). A organização hierárquica permite melhor representação das relações entre atributos, facilitando consultas que exploram essas hierarquias. Embora o aumento da complexidade das consultas seja uma consequência, o ganho em controle e qualidade dos dados justifica sua adoção em cenários específicos (Parmanto, 2005).

2.2 Amazon Athena: Arquitetura e Aplicações Analíticas

O Amazon Athena é um serviço de consulta interativa, sem servidor (*serverless*), que permite executar consultas SQL diretamente sobre dados armazenados no Amazon S3.

Utilizando o mecanismo distribuído PrestoDB/Trino, o Athena oferece alta escalabilidade e desempenho para analisar grandes volumes de dados sem necessidade de provisionamento ou gerenciamento de infraestrutura (AMAZON, 2025).

Arquitetonicamente, o Athena adota o modelo *schema-on-read*, possibilitando que os usuários definam esquemas somente no momento da leitura dos dados, o que confere grande flexibilidade para trabalhar com múltiplos formatos como CSV, JSON, ORC e Parquet. Essa abordagem facilita a integração com diferentes fontes de dados e permite consultas ad hoc e rápidas explorações analíticas, sem transformações prévias que onerem o processo (AWS, 2025).

O conceito de *schema-on-read* refere-se a uma abordagem de gerenciamento de dados na qual a estrutura ou esquema dos dados não é definido no momento em que eles são armazenados, mas apenas no momento da leitura ou análise dos mesmos. Isso permite que os dados sejam armazenados em seu formato original e bruto, oferecendo flexibilidade para trabalhar com diversos tipos de dados, incluindo dados não estruturados, semiestruturados e estruturados (SOLIX, 2024). Essa prática é fundamental para ambientes como *data lakes* e plataformas analíticas modernas, onde a definição do esquema antecipado ("*schema-on-write*") pode ser restritiva e custosa. No *schema-on-read*, o esquema é inferido dinamicamente de acordo com a necessidade da consulta, o que facilita análises exploratórias e a integração de fontes heterogêneas de dados (ELMASRI; NAVATHE, 2011). Apesar de apresentar vantagens em termos de flexibilidade e agilidade, essa abordagem pode impactar o desempenho das consultas, uma vez que o esquema precisa ser aplicado a cada leitura, demandando maior esforço computacional.

Quanto ao modelo de precificação, o Athena cobra com base no volume de dados escaneados pelas consultas, incentivando otimizações na organização dos dados, como o uso de formatos colunar e compressão para minimizar os custos e melhorar a performance (AWS, 2016).

O Athena é amplamente utilizado para pipelines analíticos que exigem baixa latência e flexibilidade, sendo ideal para consultas exploratórias, análise de logs, geração de relatórios e integração com dashboards de inteligência de negócios (HEVODATA, 2025).

Além disso, o Athena se integra nativamente com outros serviços AWS como AWS Glue para catálogo de dados e Amazon QuickSight para visualização interativa, possibilitando a construção de soluções analíticas completas baseadas em arquitetura serverless (AWS, 2025).

2.3 Redshift Serverless: Data Warehousing Serverless

O Amazon Redshift é um serviço de data warehouse em nuvem oferecido pela AWS, projetado para análise rápida e eficiente de grandes volumes de dados. Diferentemente dos bancos de dados tradicionais que processam dados linha a linha, o Redshift utiliza uma arquitetura em colunas e processamento paralelo massivo (MPP), o que otimiza o tempo de resposta em consultas analíticas complexas (ARMENATZOGLOU et al., 2022, p. 3). Essa solução é amplamente utilizada por empresas que necessitam realizar consultas complexas em seus dados para apoiar decisões estratégicas, integrando-se facilmente com ferramentas de *business intelligence* e *data lakes* como o Amazon S3.

O Amazon Redshift Serverless elimina a necessidade de configuração e gerenciamento manual de clusters. Ele provisiona e dimensiona automaticamente a capacidade do *data warehouse* conforme a demanda, assegurando alta performance mesmo em cargas de trabalho variáveis e voláteis. A cobrança é baseada no uso efetivo, medida em Redshift Processing Units por segundo, o que torna essa alternativa mais econômica e flexível para diversos perfis de uso, como projetos de testes, análises intermitentes e ambientes com flutuação de carga (FIVETRAN, 2022, p. 4).

No contexto da experiência prática apresentada neste trabalho, o Redshift Serverless foi selecionado justamente por oferecer simplicidade operacional sem comprometer a performance, permitindo focar nos aspectos de avaliação do desempenho em ambiente *serverless* sem as complexidades de administração de infraestrutura. Essa abordagem é alinhada às tendências recentes em computação na nuvem, que priorizam automação, escalabilidade dinâmica e otimização de custos (ARMENATZOGLOU et al., 2022; FIVETRAN, 2022).

2.4 EMR Serverless com Apache Spark

2.4.1 Apache Spark

O Apache Spark é uma plataforma de código aberto para processamento distribuído de grandes volumes de dados. Diferente dos modelos tradicionais, como o Hadoop MapReduce, o Spark realiza o processamento principalmente em memória, o que permite executar tarefas com

grande velocidade e eficiência (ZAHARIA et al., 2016). Sua arquitetura é baseada em clusters de computadores que trabalham de forma paralela, distribuindo os dados e as operações para otimizar o desempenho.

O núcleo do Spark, chamado Spark Core, gerencia a memória, a recuperação de falhas e a distribuição das tarefas. Sobre ele estão construídos módulos especializados, como o Spark SQL para consultas estruturadas, Spark Streaming para análise em tempo real, MLlib para aprendizado de máquina e GraphX para processamento de grafos. Essas ferramentas integradas permitem que o Spark seja uma solução completa para análises de big data, combinando rapidez, escalabilidade e flexibilidade na manipulação de dados (ZAHARIA et al., 2016; ARMENATZOGLOU et al., 2022).

2.4.2 EMR Serverless

O Amazon EMR (Elastic MapReduce) é um serviço gerenciado da AWS que facilita o processamento de grandes volumes de dados por meio de clusters configuráveis que podem executar frameworks como Apache Spark e Hadoop. Ele permite que os usuários criem, monitorem e escalem ambientes de processamento distribuído, integrando-se com outros serviços da AWS e oferecendo flexibilidade e controle sobre a infraestrutura utilizada (VERMA et al., 2025).

Já o EMR Serverless, é uma modalidade que elimina a necessidade de gerenciar os clusters diretamente. Nessa abordagem, a AWS provisiona, dimensiona e gerencia automaticamente os recursos computacionais necessários para executar as tarefas submetidas, cobrando apenas pelo uso efetivo da capacidade. Essa forma simplificada é ideal para cargas de trabalho intermitentes, prototipagem rápida e redução de custos operacionais, pois dispensa a administração de infraestrutura enquanto mantém o poder do processamento distribuído do EMR tradicional. No presente trabalho, utilizou-se o EMR Serverless para explorar essas vantagens, avaliando desempenho e usabilidade em um ambiente serverless (AWS, 2021).

2.5 Validação Científica e Referencial Acadêmico

A adoção do Star Schema Benchmark (SSB) neste trabalho não se limita à sua adequação técnica e prática, mas também se apoia em uma base sólida de validação científica. Diversos estudos revisados por pares demonstram a eficácia do SSB como ferramenta de comparação entre plataformas analíticas, especialmente em ambientes de computação em nuvem e arquiteturas serverless. Essas pesquisas evidenciam que o benchmark permite análises consistentes de desempenho, escalabilidade e eficiência, contribuindo para a identificação de características técnicas relevantes em serviços como Amazon Redshift Serverless, EMR Serverless com Apache Spark e Amazon Athena (Sanchez, 2016; Perach, Ronen & Kvatinsky, 2023).

Além disso, a literatura especializada destaca a importância da separação entre armazenamento e processamento que é característica central das soluções serverless e fator determinante para elasticidade e otimização de recursos em cenários de big data. O uso do SSB em estudos acadêmicos reforça sua confiabilidade como instrumento de avaliação, permitindo que os resultados obtidos neste trabalho estejam alinhados com práticas reconhecidas na comunidade científica e com tendências atuais em engenharia de dados e arquitetura de sistemas analíticos.

3. EXPERIMENTO

3.1 Ambiente Experimental e Ferramentas Utilizadas

O ambiente experimental foi construído integralmente sobre a infraestrutura da Amazon Web Services (AWS), com foco em serviços serverless para processamento de dados. Os testes foram realizados utilizando três plataformas distintas:

- Amazon EMR Serverless emr-7.9.0, configurado com Apache Spark para processamento distribuído de consultas;
- Amazon Redshift Serverless (PostgreSQL 8.0.2 on i686-pc-linux-gnu, compiled by GCC gcc (GCC) 3.4.2 20041017 (Red Hat 3.4.2-6.fc3), Redshift 1.0.129791), empregado como solução de data warehouse escalável e otimizada para consultas em grandes volumes de dados;
- Amazon Athena (Athena engine version 3), utilizada para execução de consultas interativas diretamente nos arquivos armazenados no Amazon S3.

Os dados de entrada foram obtidos a partir do Star Schema Benchmark (SSB), em formato TSV, e armazenados em buckets do Amazon S3. A execução das tarefas foi realizada por meio do AWS Lambda, enquanto o AWS Glue foi utilizado como catálogo de dados.

As consultas utilizadas nos experimentos seguem o modelo do *Star Schema Benchmark* (SSB) e estão apresentadas integralmente no Apêndice A.

3.2 Origem dos Dados

Todos os dados utilizados como entrada para o processamento estão armazenados no Amazon S3, em formato distribuído. Essa escolha permite que os serviços de análise acessem os dados de forma eficiente e paralela, aproveitando o modelo de armazenamento escalável da AWS.

3.3 Interação com os Serviços Serverless

A execução das consultas é realizada por meio de diferentes serviços serverless:

- Para o Amazon Athena e o Amazon Redshift Serverless, a aplicação utiliza a biblioteca Boto3, que permite a comunicação com as APIs desses serviços. O AWS Lambda é responsável por orquestrar essa interação, enviando comandos de execução e monitorando os resultados.
- Para o Amazon EMR Serverless, o processamento é feito através de um script PySpark, que contém a lógica de transformação e consulta. Esse script é submetido diretamente à aplicação EMR Serverless, que executa o código utilizando o motor de processamento Apache Spark.

3.4 Resultados do Processamento

O destino dos dados processados varia conforme o tipo de tarefa executada:

- Nas tarefas de conversão de arquivos no formato TSV para Parquet, o resultado do processamento é armazenado em um bucket do Amazon S3, garantindo que os dados transformados fiquem disponíveis para consultas e nas tarefas de execução das consultas do *benchmark*.
- Já nas tarefas das consultas do *benchmark*, a saída é visualizada diretamente pela função AWS Lambda, que pode exibir os dados ou encaminhá-los para outras aplicações. Alternativamente, os resultados e logs da execução podem ser monitorados via Amazon CloudWatch, permitindo o acompanhamento do status e desempenho das consultas em tempo real.

3.5 Arquitetura da Solução Utilizada

As Figuras 2, 3 e 4 ilustram os diagramas da arquitetura adotada para o processamento com cada serviço *serverless* na AWS, destacando os serviços utilizados em cada fase do fluxo de dados, desde a entrada dos dados até o processamento e a obtenção dos resultados, conforme detalhado nas seções anteriores.

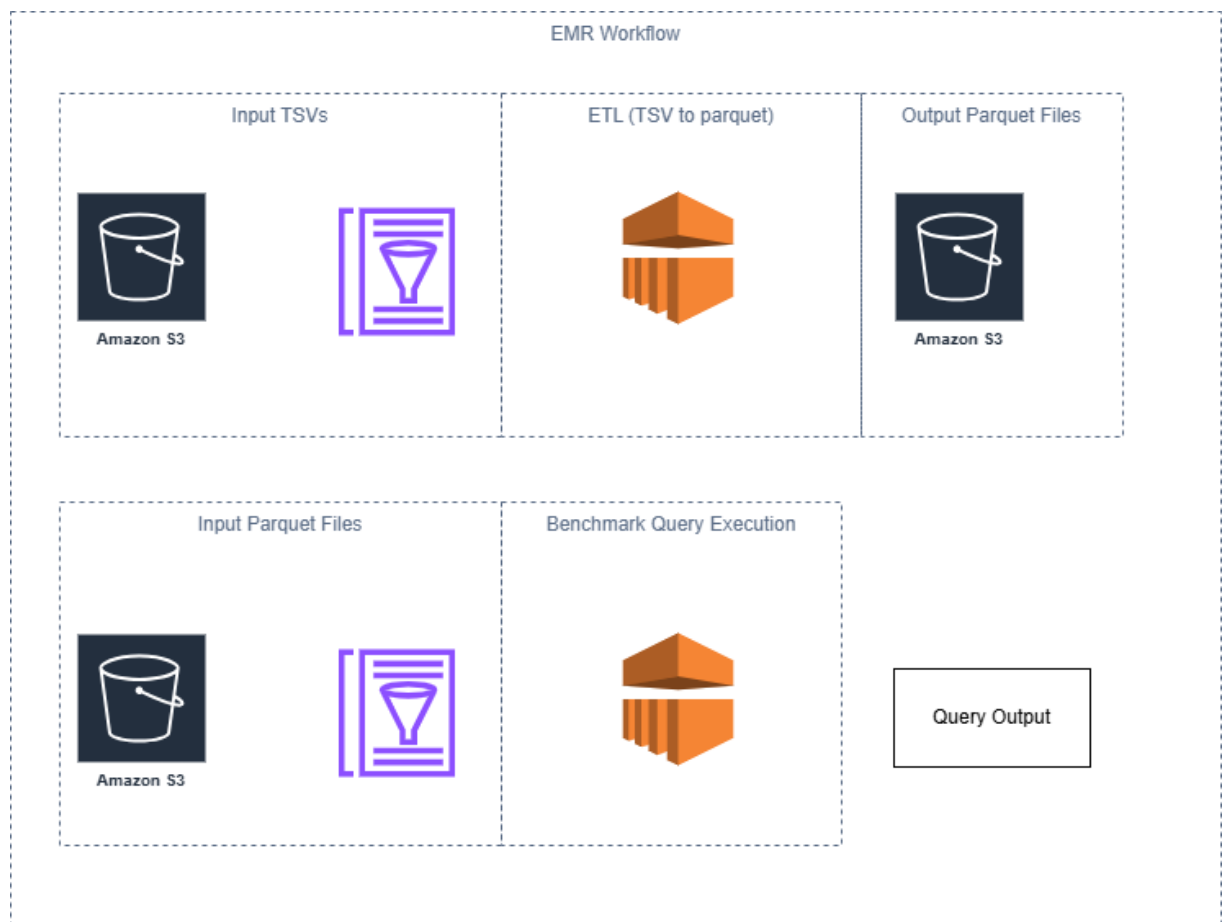


Figura 2- Diagrama ETL EMR Serverless

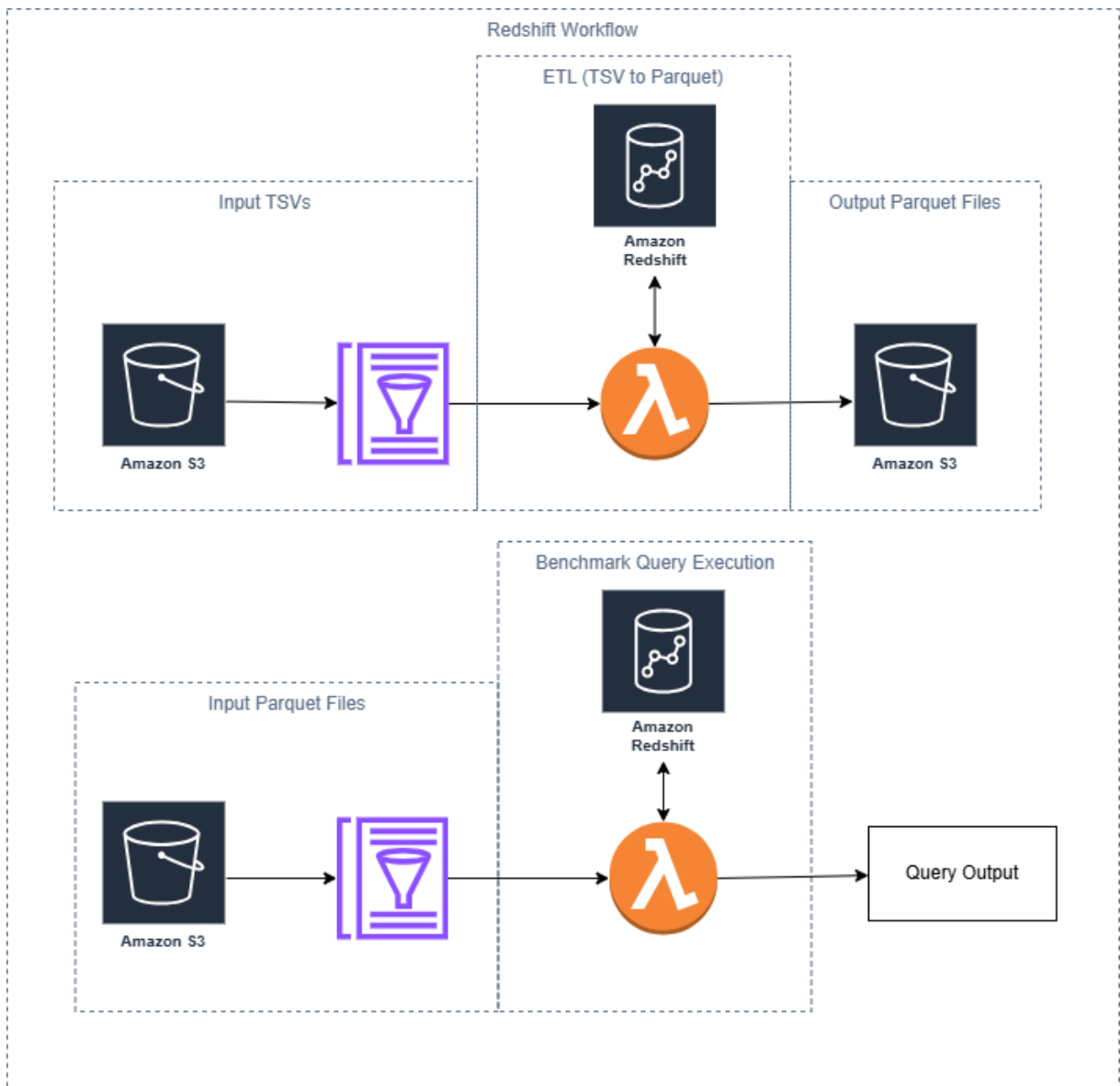


Figura 3- Diagrama ETL Redshift Serverless

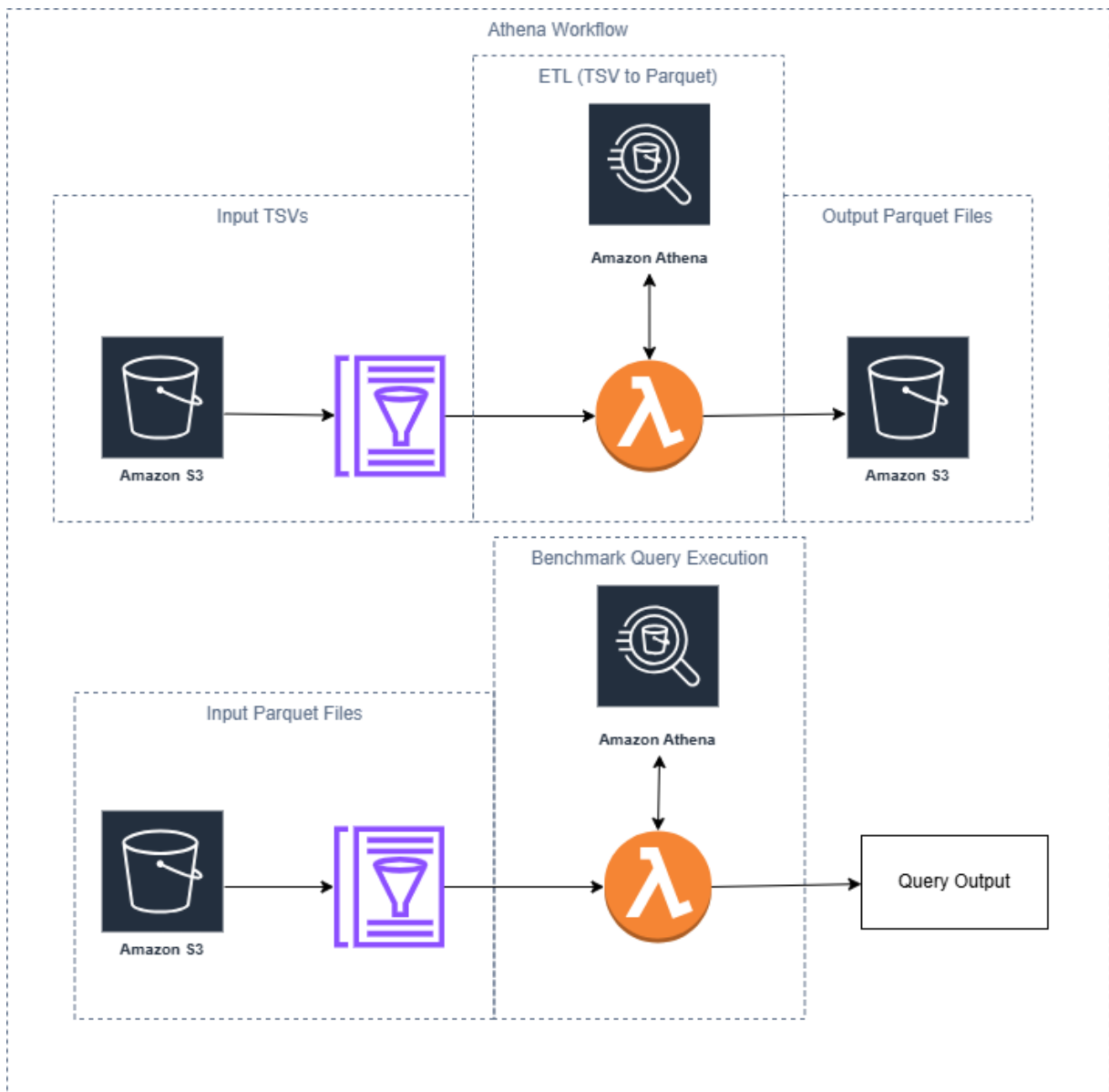


Figura 4- Diagrama ETL Athena

3.6 Implementação das ETLs e Consultas

No contexto do processamento dos dados, foram desenvolvidas duas funções AWS Lambda para interação com os serviços Amazon Athena e Amazon Redshift Serverless, enquanto o Amazon EMR Serverless utilizou *scripts* previamente armazenados no Amazon S3, que foram submetidos diretamente à aplicação para a execução dos processos de leitura, conversão e consulta.

A primeira função Lambda, implementada em Python, teve como objetivo ler os arquivos TSV do Star Schema Benchmark (SSB) armazenados no Amazon S3 e escrevê-los no formato Parquet. A segunda função foi responsável por ler os arquivos Parquet e executar as consultas do *benchmark* nos serviços Athena e Redshift Serverless.

No caso do EMR Serverless, duas aplicações PySpark distintas foram submetidas: uma dedicada à leitura dos arquivos TSV e escrita em Parquet, e outra à execução das consultas do benchmark, ambas desenvolvidas em Python e executadas sobre a mesma aplicação configurada no serviço.

O AWS Glue Data Catalog atuou como camada central de metadados, integrando os *datasets* entre as diferentes plataformas e garantindo a consistência dos esquemas de dados. Essa arquitetura híbrida, combinando funções Lambda e submissões diretas de tarefas Spark, assegurou a automação das etapas de ETL e consulta, preservando a comparabilidade entre os ambientes avaliados.

3.7 Recursos e Bibliotecas Empregadas no Processo

A implementação de todas as tarefas de processamento deste trabalho foi realizada utilizando a linguagem Python, versão 3.12, escolhida por sua ampla compatibilidade com os serviços em nuvem da AWS e pelo suporte a bibliotecas robustas voltadas à integração, manipulação e medição de desempenho de processos de dados. As bibliotecas e módulos empregados foram selecionados com base em critérios de eficiência, integração direta com os serviços utilizados e aderência às boas práticas de desenvolvimento em ambientes *serverless*.

No Amazon Redshift Serverless e Athena, foram desenvolvidos dois processos principais:

- Conversão dos arquivos TSV catalogados no Glue para o formato Parquet, executada com o uso da biblioteca boto3, responsável por estabelecer a conexão com o Redshift Serverless e Athena, realizar consultas ao AWS Glue Data Catalog e gravar os arquivos convertidos no Amazon S3. Para mensurar o desempenho, foi empregado o módulo datetime, que registrou os tempos totais de leitura e escrita.
- Execução das consultas do SSB sobre os arquivos Parquet catalogados no Glue, também com o uso do boto3 para gerenciar a conexão ao Redshift Serverless, enquanto o datetime mediu o tempo total de execução das consultas SQL.

No ambiente EMR Serverless, configurado para execução com Apache Spark, foram implementadas duas tarefas análogas por meio de arquivos contendo algoritmos em Python, armazenados no Amazon S3 e submetidos diretamente à aplicação EMR Serverless:

- Conversão dos arquivos TSV para Parquet, realizada com o módulo SparkSession da biblioteca pyspark.sql, que permitiu a criação de sessões Spark e a manipulação paralela dos dados no processo de conversão. O módulo datetime foi novamente utilizado para aferição do tempo de execução.
- Consultas SSB sobre Parquet, também utilizando o SparkSession para processamento distribuído e o datetime para mensurar o desempenho das queries.

Por fim, no serviço Amazon Athena, também implementou-se duas etapas:

- Conversão dos arquivos TSV para Parquet, utilizando a biblioteca boto3 para estabelecer a conexão e gerenciar as operações no Athena, com controle de tempo realizado pelo módulo datetime para mensurar o período total de leitura e escrita.
- Execução das consultas SSB sobre os arquivos Parquet registrados no Glue Data Catalog, com o boto3 responsável pela interface com o Athena e o datetime encarregado da medição do tempo de execução das consultas.

A Tabela a seguir apresenta os tamanhos dos arquivos utilizados nos experimentos, bem como o número de linhas contidas em cada um deles, permitindo compreender a escala dos dados processados durante os testes.

Tabela 1 - Tamanho de cada arquivo utilizado no experimento.

Nome do arquivo	Tamanho em disco (KB)	Número de linhas
customer.tbl	2,771	30,000
supplier.tbl	163	2,000
part.tbl	16,738	20,000
date.tbl	225	2,556
lineorder.tbl	585,053	6,001,171

3.8 Lógica de Execução das Consultas e Medição de Desempenho

A metodologia adotada para a execução das consultas e medição de desempenho foi dividida em quatro etapas principais, descritas a seguir:

3.8.1 Preparação e Execução Iterativa das Consultas

Inicialmente, são utilizados laços de repetição para automatizar a execução de múltiplas consultas. Cada iteração do laço representa uma tarefa de processamento, cuja entrada consiste em arquivos armazenados no Amazon S3, organizados em formato Parquet. Essa abordagem permite a execução sistemática e escalável das consultas sobre os dados distribuídos.

3.8.2 Envio das Consultas aos Serviços Serverless

As consultas são enviadas para os serviços Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless, cada um com seu respectivo mecanismo de execução:

- Para os serviços Amazon Athena e Amazon Redshift Serverless, foram utilizadas funções AWS Lambda escritas em Python, que fazem uso da biblioteca Boto3 para interagir com as APIs dos respectivos serviços
- No caso do Amazon EMR Serverless, o processamento foi realizado por meio de um script Python desenvolvido com a biblioteca PySpark, armazenado no Amazon S3 e submetido diretamente à aplicação. A biblioteca PySpark fornece uma interface com o motor de processamento Apache Spark.

3.8.3 Monitoramento da Execução

Antes do envio da consulta, é registrado o *timestamp* inicial utilizando o módulo `datetime`. Após o envio, a execução é monitorada por meio das APIs dos serviços, que informam o status da operação. Quando o status `SUCCEEDED` é recebido, indicando que a consulta foi concluída com sucesso, é registrado um novo timestamp, representando o momento final da execução.

3.8.4 Cálculo do Tempo de Execução

Com os dois *timestamps* registrados (inicial e final), é realizada a subtração entre os tempos, resultando no tempo total de execução da consulta, medido em segundos. Esse valor é armazenado em um objeto do tipo dicionário, onde a chave representa o identificador da

consulta executada e o valor corresponde ao tempo de execução. Essa estrutura permite a organização e posterior análise comparativa do desempenho entre os diferentes serviços e consultas.

4. RESULTADOS E COMENTÁRIOS

4.1 Transformação de Dados

A Tabela 2 apresenta os resultados obtidos na etapa de transformação dos dados, indicando o tempo médio necessário para realizar a leitura dos arquivos no formato TSV e a escrita no formato Parquet para cada tabela do benchmark. Os valores correspondem à média de cinco execuções por operação, realizadas com intervalos mínimos de 24 horas para evitar interferência de cache, e refletem o desempenho dos três serviços serverless avaliados: Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless com Spark.

Tabela 2 – Tempo médio de leitura em segundos (TSV) + escrita (Parquet) por tabela (média de 5 amostras):

Dataset	Athena	Redshift	EMR
customer	1.7604	1.6929	5.2088
date	1.5814	1.6755	2.2201
lineorder	5.6200	4.8096	12.2612
part	2.1699	2.0363	2.0712
supplier	1.4756	1.4035	1.5120

A Figura 5 apresenta a comparação dos tempos médios obtidos na transformação de cada arquivo, considerando a leitura no formato TSV e a escrita no formato Parquet. O gráfico evidencia as diferenças de desempenho entre os três serviços serverless avaliados: Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless com Spark, com base na média das cinco execuções realizadas para cada arquivo e serviço.

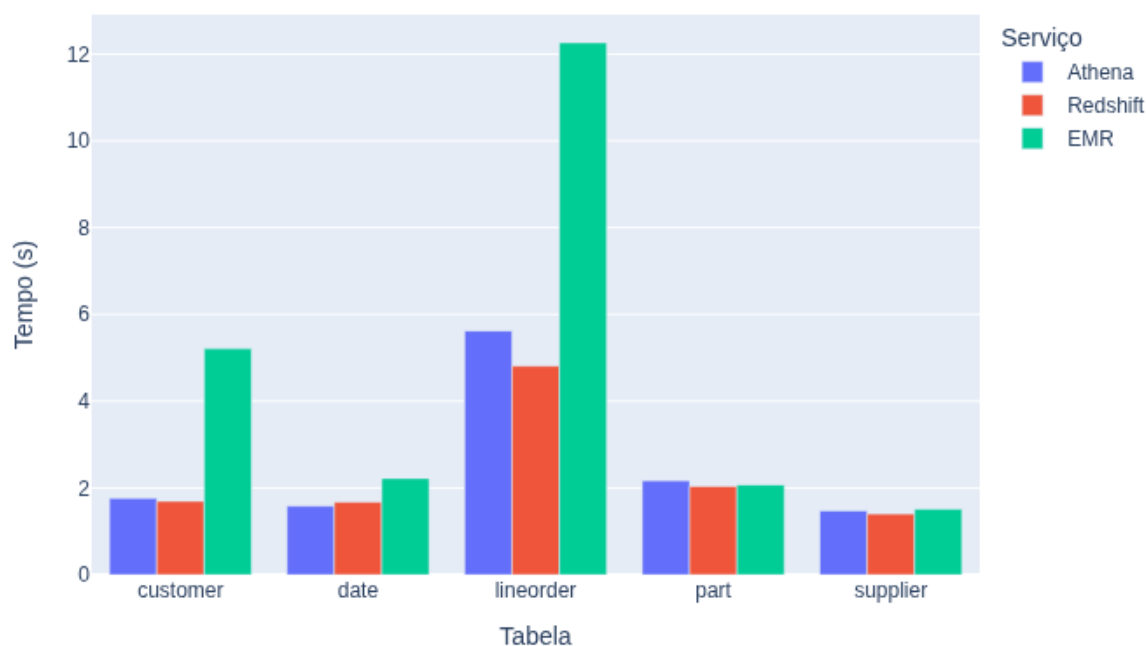


Figura 5 – Gráfico comparando tempos médios das 5 amostras de leitura TSV + escrita Parquet por serviço

Tempos médio por serviço:

- Athena: 2.52 s
- Redshift: 2.32 s
- EMR: 4.65 s

O Redshift Serverless apresentou o melhor desempenho geral na transformação, seguido de perto pelo Athena. O EMR Serverless foi significativamente mais lento, especialmente na tabela 'lineorder', evidenciando o impacto do *overhead* de inicialização do Spark.

4.2 Execução das Consultas no Formato Parquet

As consultas do SSB foram executadas sobre os arquivos Parquet utilizando os mesmos três serviços *serverless*. Os tempos apresentados correspondem à média de cinco execuções por consulta.

A Tabela 3 apresenta os tempos médios de execução das consultas realizadas sobre os arquivos no formato Parquet, considerando as três soluções serverless avaliadas: Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless com Spark. Os valores correspondem à média de cinco execuções por consulta, permitindo comparar o desempenho dos serviços na etapa de processamento analítico após a transformação dos dados.

Tabela 3 – Tempo médio em segundos de execução das queries (Parquet):

Query	Athena	Redshift	EMR
Q1.1	1.4425	1.6171	0.5027
Q1.2	1.5238	1.4879	0.6043
Q1.3	1.5631	1.4704	0.3412
Q2.1	2.0945	1.9409	1.0219
Q2.2	1.8390	1.6036	0.6018
Q2.3	1.8039	1.6344	0.6193
Q3.1	2.1251	1.6753	0.6386
Q3.2	1.8151	1.6022	0.6003
Q3.3	1.8495	1.5273	0.5952
Q3.4	1.8509	1.5442	0.5615
Q4.1	2.2364	1.7996	0.7124
Q4.2	2.1717	1.8846	0.7303
Q4.3	2.6896	1.8081	0.6895

A Figura 6 apresenta a comparação dos tempos médios de execução das consultas do *Star Schema Benchmark* (SSB) entre os três serviços serverless avaliados: Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless com Spark. O gráfico evidencia as diferenças de desempenho, considerando a média das cinco execuções realizadas para cada serviço, permitindo uma análise clara da eficiência relativa de cada solução.

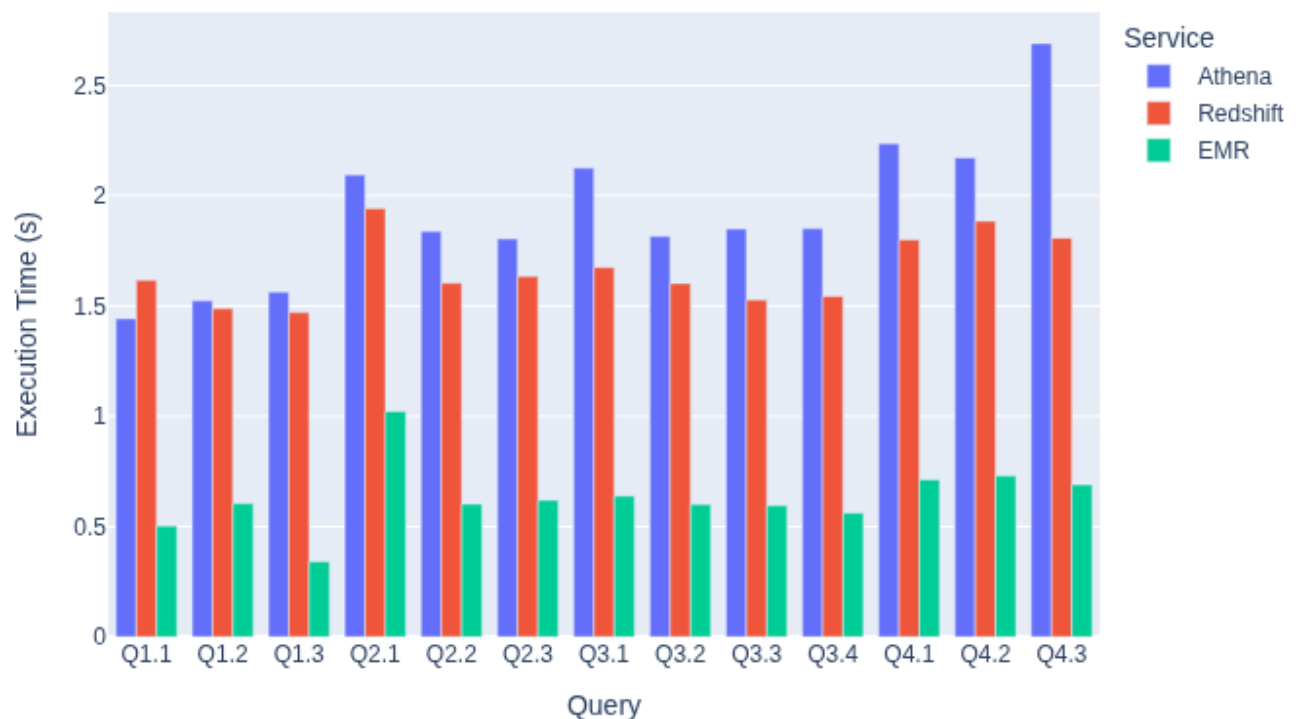


Figura 6– Comparação dos tempos médios das 5 amostras de execução das consultas SSB por serviço

Médias gerais por serviço:

- Athena: 1.923 s
- Redshift: 1.661 s
- EMR: 0.632 s

O EMR Serverless apresentou desempenho significativamente superior na execução das consultas, sendo em média 62% mais rápido que o Redshift. Isso contrasta com seu desempenho inferior na etapa de transformação, indicando que a sobre carga de trabalho inicial do Spark é compensado em processamento sobre dados otimizados.

4.3 Resultados

Os resultados obtidos revelam comportamentos distintos entre os serviços serverless da AWS. Na etapa de transformação de dados (TSV para Parquet), o Redshift Serverless demonstrou maior eficiência, seguido pelo Athena. O EMR Serverless, embora tenha apresentado maior tempo nessa fase, destacou-se na execução das consultas sobre os arquivos

Parquet, sendo o mais rápido em todas as queries do benchmark SSB. Isso evidencia que o EMR Serverless é mais adequado para cargas de trabalho complexas, enquanto Athena e Redshift são mais indicados para tarefas interativas e pipelines leves.

A comparação entre os serviços Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless demonstra que a escolha da tecnologia deve considerar o tipo de carga de trabalho e os objetivos do *pipeline* de dados.

Na etapa de transformação de dados (TSV → Parquet), o Redshift Serverless apresentou o melhor desempenho médio (2,32 s), seguido pelo Athena (2,52 s), enquanto o EMR Serverless foi mais lento (4,65 s), principalmente devido ao overhead de inicialização do Spark. Esse comportamento confirma que soluções orientadas a SQL, como Redshift e Athena, são mais adequadas para tarefas de ETL leves e interativas, onde a latência inicial impacta diretamente o tempo total de execução.

Por outro lado, na execução das consultas analíticas sobre arquivos Parquet, o EMR Serverless demonstrou superioridade, com tempo médio de 0,632 s — cerca de 62% mais rápido que o Redshift (1,661 s) e 67% mais rápido que o Athena (1,923 s). Esse resultado indica que, apesar do custo inicial de inicialização, arquiteturas baseadas em Spark são mais eficientes para cargas de trabalho complexas e consultas intensivas, aproveitando paralelismo e otimizações de processamento distribuído.

Um aspecto adicional observado foi que o EMR Serverless particionou os arquivos Parquet em um número maior de objetos, consequência do modelo de paralelismo do Spark. Essa estratégia aumenta a capacidade de leitura paralela, mas pode introduzir o problema dos “*small files*”, que impacta negativamente mecanismos de metadados e eficiência de leitura em sistemas como Athena e Redshift Spectrum. A literatura e as boas práticas da AWS recomendam manter arquivos Parquet entre 128 MB e 1 GB e evitar excesso de partições, pois isso reduz varreduras desnecessárias e melhora o desempenho global.

Portanto, conclui-se que Athena e Redshift Serverless são recomendados para pipelines de transformação e consultas interativas, enquanto o EMR Serverless é mais indicado para cenários de análise massiva e processamento distribuído, desde que se gerencie adequadamente o número e o tamanho dos arquivos Parquet. Essa constatação reforça que não existe uma solução universal. A escolha depende do perfil da aplicação, volume de dados e requisitos de desempenho.

5 CONCLUSÃO

5.1 Principais Constatações e Contribuições

Este trabalho realizou uma análise comparativa sistemática do desempenho de três serviços serverless da AWS: Amazon Athena, Amazon Redshift Serverless e Amazon EMR Serverless, por meio da execução de cargas de trabalho baseadas no Star Schema Benchmark. Os resultados revelaram padrões distintos de desempenho, com implicações relevantes para a arquitetura de pipelines de dados em ambientes de computação em nuvem.

O principal achado está na dicotomia observada entre as etapas de transformação e consulta de dados. Na conversão de arquivos TSV para o formato Parquet, o Redshift Serverless apresentou o melhor desempenho, com tempo médio de 2,32 segundos, seguido pelo Athena (2,52s). O EMR Serverless, por sua vez, foi significativamente mais lento (4,65s), devido ao overhead de inicialização do Spark, que impacta negativamente operações pontuais de transformação.

Em contraste, na execução de consultas analíticas sobre dados em Parquet, o EMR Serverless demonstrou desempenho superior, com tempo médio de 0,632 segundos — 62% mais rápido que o Redshift (1,661s) e 67% mais rápido que o Athena (1,923s). Essa inversão de performance evidencia que a arquitetura distribuída do Spark e seu processamento em memória são particularmente eficazes para operações analíticas complexas, nas quais o paralelismo massivo pode ser explorado com maior eficiência.

5.2 Méritos do Estudo

Este trabalho apresenta várias contribuições metodológicas e práticas para a área de engenharia de dados:

- **Rigor Metodológico:** A utilização do Star Schema Benchmark (SSB) como ferramenta de avaliação proporcionou uma base consistente e reproduzível para comparação, seguindo práticas consolidadas na literatura científica. A execução de múltiplas iterações para cada operação garantiu a confiabilidade estatística dos resultados.

- **Abordagem Prática:** A implementação em ambiente real da AWS, utilizando configurações padrão dos serviços, assegurou que os resultados refletissem cenários práticos enfrentados por profissionais de engenharia de dados. A arquitetura serverless adotada representa o estado da arte em processamento de dados em nuvem.
- **Análise Multidimensional:** A avaliação contemplou tanto aspectos de desempenho puro quanto características operacionais, proporcionando uma visão holística das vantagens e limitações de cada serviço.

5.3 Limitações e Restrições

É importante contextualizar os resultados dentro das limitações inerentes ao escopo do estudo:

- **Cargas de Trabalho Específicas:** A análise focou exclusivamente em cargas de trabalho OLAP, não contemplando cenários de processamento em tempo real, streaming ou operações de machine learning. Diferentes perfis de cargas de trabalho podem revelar comportamentos distintos dos serviços avaliados.
- **Otimizações Avançadas:** O estudo utilizou configurações padrão dos serviços, não explorando otimizações avançadas que poderiam melhorar o desempenho individual de cada plataforma. Técnicas como particionamento customizado, compactação otimizada ou ajuste de parâmetros específicos podem alterar os resultados observados.
- **Aspectos Econômicos:** A análise restringiu-se a métricas de desempenho, não incorporando uma avaliação detalhada de custo-benefício que, em cenários reais, frequentemente influencia a seleção de tecnologias.
- **Escala Limitada:** Embora o volume de dados utilizado seja representativo para muitos cenários práticos, resultados em escala petabyte podem revelar comportamentos adicionais não capturados neste experimento.

5.4 Trabalhos Futuros

Os achados deste trabalho abrem várias perspectivas para investigações futuras:

- **Análise de Custo-Benefício:** Um estudo complementar incorporando métricas econômicas detalhadas permitiria uma avaliação mais completa do valor proporcionado por cada serviço em diferentes cenários de uso.

- **Cargas de Trabalho Híbridas:** A investigação do desempenho em cargas de trabalho que combinam características OLTP e OLAP, representativas de aplicações modernas de dados, constitui uma direção promissora.
- **Otimizações Avançadas:** Pesquisas focadas em técnicas de otimização específicas para cada serviço poderiam quantificar os ganhos de performance alcançáveis através de configurações customizadas.
- **Escala Extendida:** A reprodução do experimento em volumes de dados significativamente maiores permitiria avaliar o comportamento de escalabilidade dos serviços em condições extremas.
- **Arquiteturas Híbridas:** O desenvolvimento e avaliação de arquiteturas que combinam múltiplos serviços de forma complementar representa uma oportunidade para maximizar as vantagens de cada plataforma.

5.5 Contribuições para a Prática Profissional

Este trabalho oferece contribuições tangíveis para profissionais e organizações que utilizam ou avaliam serviços serverless AWS para processamento analítico de dados, desta forma os resultados fornecem um framework baseado em evidências para a seleção de tecnologias serverless conforme características específicas do workload, distinguindo claramente entre cenários de transformação leve e análise complexa.

Ademais, as descobertas sobre o comportamento do EMR Serverless na geração de arquivos Parquet alertam para a importância do gerenciamento adequado de metadados de mapeamento e particionamento, informando melhores práticas de arquitetura de dados.

Por fim, o estudo evidencia as compensações entre latência inicial e performance sustentada, destacando a existência de sobrecarga inicial que, embora não seja considerada no modelo, é relevante para que arquitetos estejam cientes ao avaliar cenários que envolvem ganhos de longo prazo.

Esta pesquisa reforça que a escolha entre serviços serverless AWS deve ser guiada por uma compreensão profunda das características específicas de cada carga de trabalho, não existindo uma solução universalmente superior. A combinação inteligente dessas tecnologias, aproveitando suas fortalezas complementares, representa a abordagem mais promissora para a construção de pipelines de dados eficientes e econômicos em ambientes cloud modernos.

6 REFERÊNCIAS BIBLIOGRÁFICAS

AMAZON WEB SERVICES. Amazon Athena – query service. White paper. Seattle: AWS, 2025. Disponível em: <https://docs.aws.amazon.com/athena/>. Acesso em: 20 out. 2025.

AMAZON WEB SERVICES. Big Data Analytics Options on AWS. White paper, July 26 2021. Disponível em: <https://docs.aws.amazon.com/whitepapers/latest/big-data-analytics-options/welcome.html>. Acesso em: 25 out. 2025.

AMAZON WEB SERVICES. Overview of Amazon Web Services. White paper, rev. 9 jun. 2025. Disponível em: <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/introduction.html>. Acesso em: 15 out. 2025.

ARMENATZOGLOU, N.; BASU, S.; BHANOORI, N.; et al. Amazon Redshift re-invented. In: INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA (SIGMOD '22), Philadelphia, PA, USA, 12-17 June 2022. New York: ACM, 2022. p. 1-13. DOI: 10.1145/3514221.3526045. Disponível em: <https://assets.amazon.science/93/e0/a347021a4c6fbbccd5a056580d00/sigmod22-redshift-reinvented.pdf>. Acesso em: 19 out. 2025.

BALDINI, I.; CASTRO, P.; CHANG, K.; CHENG, P.; FINK, S.; ISHAKIAN, V.; MITCHELL, N.; MUTHUSAMY, V.; RABBAH, R.; SLOMINSKI, A.; SUTER, P. Serverless Computing: Current Trends and Open Problems. In: Research Advances in Cloud Computing. Singapore: Springer Singapore, 2017. p. 1-20. DOI: 10.1007/978-981-10-5026-8_1. Disponível em: <https://arxiv.org/abs/1706.03178>. Acesso em: 29 out. 2025.

EWAI, M.; CHOW, P. Disaggregated Memory Systems: Challenges and Opportunities. IEEE Micro, v. 43, n. 1, p. 52-61, 2023. DOI: 10.1109/ACCESS.2023.3250407. Disponível em: <https://ieeexplore.ieee.org/document/10056149/citations#citations>. Acesso em 18 out. 2025.

FIVETRAN. Fivetran named in 2022 Gartner Magic Quadrant and Market Share Analysis reports. Oakland: Fivetran, 8 set. 2022. Disponível em: <https://www.fivetran.com/blog/i-took-redshift-serverless-for-a-spin-heres-what-i-found>. Acesso em: 10 out. 2025.

VERMA, A. et al. Managed resource scaling in Amazon EMR. Amazon Science, 2025. Disponível em: <https://www.amazon.science/publications/managed-resource-scaling-in-amazon-emr>. Acesso em: 15 dez. 2025.

GHORBAN, M.; GHOBAEI-ARANI, M.; ESMAEILI, L. A survey on the scheduling mechanisms in serverless computing: a taxonomy, challenges, and trends. *Cluster Computing*, v. 27, 2024. DOI: 10.1007/s10586-023-04264-8. Disponível em: <https://link.springer.com/article/10.1007/s10586-023-04264-8>. Acesso em: 10 out. 2025.

GRAY, J. *The Benchmark Handbook for Database and Transaction Processing Systems*. 2. ed. San Francisco: Morgan Kaufmann Publishers, 1993. Disponível em: <https://archive.org/details/transactionproce0000gray/page/n5/mode/2up>. Acesso em: 20 out. 2025.

HAMEED, S.; et al. A comparative analysis on the services offered by cloud computing providers: AWS, Azure and GCP. *Concurrency and Computation: Practice and Experience*, v. 37, n. 17, e70215, 2025. Disponível em: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.70215>. Acesso em: 13 out. 2025.

HEVODATA. The Future of Data Engineering Is Here — 5 Trends You Can't Ignore in 2025! Hevo Data Blog, 9 jan. 2025. Disponível em: <https://hevodata.com/learn/2025-data-engineering-predictions/>. Acesso em: 20 out. 2025.

HOKAMA, D. D. B. et al. *A modelagem de dados no ambiente Data Warehouse*. São Paulo: Universidade Presbiteriana Mackenzie, Faculdade de Computação e Informática, 2004. Trabalho de Graduação (Bacharelado em Sistemas de Informação). Disponível em: <http://meusite.mackenzie.com.br/rogerio/tgi/2004ModelagemDW.pdf>. Acesso em: 13 out. 2025.

KIMBALL, R.; ROSS, M. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3. ed. Hoboken, NJ: John Wiley & Sons, 2013. ISBN 978-1118530801. Disponível em: <https://www.wiley.com/en-us/The+Data+Warehouse+Toolkit%3A+The+Definitive+Guide+to+Dimensional+Modeling%2C+3rd+Edition-p-9781118530801>. Acesso em: 22 out. 2025.

KOUNEV, S.; HERBST, N.; ABAD, C. L.; IOSUP, A.; FOSTER, I.; SHENOY, P.; RANA, O.; CHIEN, A. A. Serverless Computing: What It Is, and What It Is Not? *Communications of the ACM*, v. 66, n. 9, p. 80-92, 2023. DOI: 10.1145/3587249. Disponível em: <https://dl.acm.org/doi/10.1145/3587249>. Acesso em: 20 out. 2025.

NAKAR, O. Running SQL on Amazon Athena to Analyze Big Data Quickly and across Regions. *AWS Blogs*, 28 abr. 2020. Disponível em: <https://aws.amazon.com/blogs/apn/running-sql-on-amazon-athena-to-analyze-big-data-quickly-and-across-regions/>. Acesso em: 13 out. 2025.

NOUHAS, H.; et al. Comparing AWS, Azure, and Google Cloud Platform. *International Journal of Engineering and Technology Trends (IJETT)*, v. 73, n. 6, p. 350-370, jun. 2025. Disponível em: <https://ijettjournal.org/Volume-73/Issue-6/IJETT-V73I6P129.pdf>. Acesso em: 20 out. 2025.

O'NEIL, P.; O'NEIL, E.; CHEN, X. The Star Schema Benchmark (SSB). OLAP Council and UMASS/Boston, 2009. Disponível em: <http://www.cs.umb.edu/~poneil/StarSchemaB.PDF>. Acesso em: 29 out. 2025.

PARMANTO, B.; SCOTCH, M.; AHMAD, S. A Framework for Designing a Healthcare Outcome Data Warehouse. *Perspectives in Health Information Management*, v. 2, n. 3, Summer 2005. Disponível em: https://www.researchgate.net/publication/5782658_A_Framework_for_Designing_a_Healthcare_Outcome_Data_Warehouse. Acesso em: 19 out. 2025.

PERACH, B.; RONEN, R.; KVATINSKY, S. Understanding Bulk-Bitwise Processing In-Memory Through Database Analytics. In: 2023 IEEE/ACM International Symposium on High-Performance Computer Architecture (HPCA). New York: IEEE/ACM, 2024. p. 705-717. DOI: 10.1109/TETC.2023.3315189. Disponível em: <https://ieeexplore.ieee.org/document/10255629/authors#authors>. Acesso em: 17 out. 2025.

SADAQAT, M.; SÁNCHEZ-GORDÓN, M.; COLOMO-PALACIOS, R. Benchmarking Serverless Computing: Performance and Usability. *Journal of Information Technology Research*, v. 15, n. 1, p. 1-20, 2022. Disponível em: <https://www.igi-global.com/article/benchmarking-serverless-computing-performance-and-usability/299374>. Acesso em: 18 out. 2025.

SCHMID, L.; COPIK, M.; CALOTOIU, A.; et al. SeBS-Flow: Benchmarking Serverless Cloud Function Workflows. In: EuroSys '25 – Twentieth European Conference on Computer Systems, 2025. DOI: 10.1145/3689031.3717465. Disponível em: <https://dl.acm.org/doi/epdf/10.1145/3689031.3717465>. Acesso em: 19 out. 2025.

SOLIX TECHNOLOGIES, INC. What Is a Data Lake and Why Does It Matter in 2024? Santa Clara, CA: Solix Technologies, 2024. Disponível em: <https://www.solix.com/products/answers/what-is-a-data-lake/>. Acesso em: 17 out. 2025.

TOOSI, A. N.; JAVADI, B.; IOSUP, A.; SMIRNI, E.; DUSTDAR, S. Serverless Computing for Next-generation Application Development. *Future Generation Computer Systems*, v. 164, p. 107573, 2025. DOI: 10.1016/j.future.2024.107573. Disponível em: [https://doi.org/10.59324/ejtas.2023.1\(5\).25.25](https://doi.org/10.59324/ejtas.2023.1(5).25.25)). Acesso em: 19 out.2025.

WEN, J.; CHEN, Z.; SARRO, F.; LIU, X. Unveiling overlooked performance variance in serverless computing. arXiv preprint arXiv:2305.04309, 2023. Disponível em: <https://arxiv.org/abs/2305.04309>. Acesso em: 19 out. 2025.

WEN, J.; LIU, Y.; CHEN, Z.; CHEN, J.; MA, Y. Characterizing Commodity Serverless Computing Platforms. Software: Practice and Experience, 2023. DOI: 10.1002/smr.2394. Disponível em: <https://doi.org/10.1002/smr.2394>. Acesso em: 20 out. 2025

ZAHARIA, M.; XIN, R. S.; WENDELL, P.; DAS, T.; ARMBRUST, M.; DAVE, A.; MENG, X.; ROSEN, J.; VENKATARAMAN, S.; FRANKLIN, M. J.; GHODSI, A.; GONZALEZ, J.; SHENKER, S.; STOICA, I. Apache Spark: A Unified Engine for Big Data Processing. Communications of the ACM, v. 59, n. 11, p. 56-65, nov. 2016. DOI: 10.1145/2934664. Disponível em: <https://dl.acm.org/doi/10.1145/2934664>. Acesso em: 22 out. 2025.

7 APÊNDICE A - CONSULTAS UTILIZADAS NO BENCHMARK

A seguir, são apresentadas as consultas SQL utilizadas para a execução dos experimentos descritos neste trabalho. As consultas foram executadas sobre o dataset Star Schema Benchmark (SSB) armazenado no formato Parquet no ambiente EMR.

7.1 Consulta q1.1

```
SELECT SUM(extended_price * discount) AS revenue
FROM glue_catalog.lineorder
WHERE discount BETWEEN 1 AND 3
AND quantity < 25
AND order_date BETWEEN 19930101 AND 19931231;
```

7.2 Consulta q1.2

```
SELECT SUM(lo.extended_price * lo.discount) AS revenue
FROM glue_catalog.lineorder lo
JOIN glue_catalog.date d
ON lo.order_date = d.date_key
WHERE d.year = 1993
AND lo.discount BETWEEN 1 AND 3
AND lo.quantity < 25;
```

7.3 Consulta q1.3

```
SELECT SUM(lo.extended_price * lo.discount) AS revenue
FROM glue_catalog.lineorder lo
JOIN glue_catalog.date d ON lo.order_date = d.date_key
WHERE d.year = 1994
AND lo.discount BETWEEN 4 AND 6
AND lo.quantity BETWEEN 26 AND 35;
```

7.4 Consulta q2.1

```
SELECT SUM(lo.revenue) AS revenue, d.year, p.brand1
FROM glue_catalog.lineorder lo
JOIN glue_catalog.date d ON lo.order_date = d.date_key
JOIN glue_catalog.part p ON lo.part_key = p.part_key
JOIN glue_catalog.supplier s ON lo.supp_key = s.supp_key
```

```

WHERE p.category = 'MFGR#12'
  AND s.region_name = 'AMERICA'
GROUP BY d.year, p.brand1
ORDER BY d.year, p.brand1;

```

7.5 Consulta q2.2

```

SELECT SUM(lo.revenue) AS revenue, d.year, p.brand1
FROM glue_catalog.lineorder lo
JOIN glue_catalog.date d    ON lo.order_date = d.date_key
JOIN glue_catalog.part p    ON lo.part_key = p.part_key
JOIN glue_catalog.supplier s ON lo.supp_key = s.supp_key
WHERE p.brand1 BETWEEN 'MFGR#2221' AND 'MFGR#2228'
  AND s.region_name = 'ASIA'
GROUP BY d.year, p.brand1
ORDER BY d.year, p.brand1;

```

7.6 Consulta q2.3

```

SELECT SUM(lo.revenue) AS revenue, d.year, p.brand1
FROM glue_catalog.lineorder lo
JOIN glue_catalog.date d    ON lo.order_date = d.date_key
JOIN glue_catalog.part p    ON lo.part_key = p.part_key
JOIN glue_catalog.supplier s ON lo.supp_key = s.supp_key
WHERE p.brand1 = 'MFGR#2221'
  AND s.region_name = 'EUROPE'
GROUP BY d.year, p.brand1
ORDER BY d.year, p.brand1;

```

7.7 Consulta q3.1

```

SELECT c.c_nation, s.nation, d.year, SUM(lo.revenue) AS revenue
FROM glue_catalog.customer c
JOIN glue_catalog.lineorder lo ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s  ON lo.supp_key = s.supp_key
JOIN glue_catalog.date d      ON lo.order_date = d.date_key
WHERE c.c_region = 'ASIA'
  AND s.region_name = 'ASIA'
  AND d.year BETWEEN 1992 AND 1997
GROUP BY c.c_nation, s.nation, d.year
ORDER BY d.year ASC, revenue DESC;

```

7.8 Consulta q3.2

```
SELECT c.c_city, s.region, d.year, SUM(lo.revenue) AS revenue
FROM glue_catalog.customer c
JOIN glue_catalog.lineorder lo ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s ON lo.supp_key = s.supp_key
JOIN glue_catalog.date d ON lo.order_date = d.date_key
WHERE c.c_nation = 'UNITED STATES'
AND s.region = 'UNITED STATES'
AND d.year BETWEEN 1992 AND 1997
GROUP BY c.c_city, s.region, d.year
ORDER BY d.year ASC, revenue DESC;
```

7.9 Consulta q3.3

```
SELECT c.c_city, s.nation, d.year, SUM(lo.revenue) AS revenue
FROM ssb_emr.glue_catalog.customer c
JOIN glue_catalog.lineorder lo ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s ON lo.supp_key = s.supp_key
JOIN glue_catalog.date d ON lo.order_date = d.date_key
WHERE c.c_city IN ('UNITED K11', 'UNITED K15')
AND s.nation IN ('UNITED K11', 'UNITED K15')
AND d.year BETWEEN 1992 AND 1997
GROUP BY c.c_city, s.nation, d.year
ORDER BY d.year ASC, revenue DESC;
```

7.10 Consulta q3.4

```
SELECT c.c_city, s.nation, d.year, SUM(lo.revenue) AS revenue
FROM glue_catalog.customer c
JOIN glue_catalog.lineorder lo ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s ON lo.supp_key = s.supp_key
JOIN glue_catalog.date d ON lo.order_date = d.date_key
WHERE c.c_city IN ('UNITED K11', 'UNITED K15')
AND s.nation IN ('UNITED K11', 'UNITED K15')
AND d.year_month = 'Dec1997'
GROUP BY c.c_city, s.nation, d.year
ORDER BY d.year ASC, revenue DESC;
```

7.11 Consulta q4.1

```
SELECT d.year, c.c_nation,
SUM(lo.revenue - lo.supply_cost) AS profit
FROM glue_catalog.date d
```

```

JOIN glue_catalog.lineorder lo  ON lo.order_date = d.date_key
JOIN glue_catalog.customer c    ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s    ON lo.supp_key = s.supp_key
JOIN glue_catalog.part p        ON lo.part_key = p.part_key
WHERE c.c_region = 'AMERICA'
      AND s.region_name = 'AMERICA'
      AND p.mfgr IN ('MFGR#1','MFGR#2')
GROUP BY d.year, c.c_nation
ORDER BY d.year, c.c_nation;

```

7.12 Consulta q4.2

```

SELECT d.year, s.region, p.category,
       SUM(lo.revenue - lo.supply_cost) AS profit
FROM glue_catalog.date d
JOIN glue_catalog.lineorder lo  ON lo.order_date = d.date_key
JOIN glue_catalog.customer c    ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s    ON lo.supp_key = s.supp_key
JOIN glue_catalog.part p        ON lo.part_key = p.part_key
WHERE c.c_region = 'AMERICA'
      AND s.region_name = 'AMERICA'
      AND d.year IN (1997, 1998)
      AND p.mfgr IN ('MFGR#1','MFGR#2')
GROUP BY d.year, s.region, p.category
ORDER BY d.year, s.region, p.category;

```

7.13 Consulta q4.3

```

SELECT d.year, s.nation, p.brand1,
       SUM(lo.revenue - lo.supply_cost) AS profit
FROM glue_catalog.date d
JOIN glue_catalog.lineorder lo ON lo.order_date = d.date_key
JOIN glue_catalog.customer c  ON lo.cust_key = c.c_custkey
JOIN glue_catalog.supplier s  ON lo.supp_key = s.supp_key
JOIN glue_catalog.part p      ON lo.part_key = p.part_key
WHERE c.c_region = 'AMERICA'
      AND s.region = 'UNITED STATES'
      AND d.year IN (1997, 1998)
      AND p.category = 'MFGR#14'
GROUP BY d.year, s.nation, p.brand1
ORDER BY d.year, s.nation, p.brand1;

```